SIMATS ENGINEERING

CSA15 – CLOUD COMPUTING AND BIG DATA ANALYTICS

ASSIGNMENT 3 – CO3
OBE-Based Evidence Principle

Design a Cloud-Based Collaborative Solution
CO3: *Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications.*

Field	Details	Field	Details
Student Name	KENISHA JUDITH J	Register Number	192572071
Department / Section	CSE(AI)	Academic Year / Semester	2ND
Faculty Name	Dr.ArulRaj A M	Nature of Assignment	Individual
Duration	1 Week	Total Marks	50
Submission Date		Evaluation Date	

MARK SHEET (Rubric-based evaluation)

Task	Title	Max	Awarded	Evidence / evaluation focus
1	Accessing Cloud Services through Different Methods	6		Access-method diagram; demonstration; screenshots; comparison
2	Cloud Platforms and Web Applications	7		Two platforms; interaction diagram; app demonstration; evaluation
3	Web APIs for Cloud Access	7		API design; executable client; two requests; responses
4	Cloud Security and Network Access	7		Secure-access architecture; threats; authentication; mitigations
5	Collaboration Using Cloud-Based Services	6		Shared workspace; permissions; edits; contribution evidence
6	Integrated Cloud Access and Collaboration Mini Investigation	7		Scenario; architecture; prototype; two user tests; evaluation
7	Day-4 Design Challenge: Cloud-Based Collaborative Solution	10		Two independent proposals; comparison; joint design; validation; defence
	TOTAL	**50**		**Percentage:**

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = Present in documented answer; P = Partly present; ✗ = Missing / not satisfied

Task	Evidence expected	Status	Evaluator note
1	Three access methods, client–network–cloud diagram, demonstration and observations		Access-method classification and practical access screenshots to be checked.
2	Two-platform comparison, interaction diagram and cloud web-app demonstration		Platform mapping, user-action sequence and local-versus-cloud evaluation to be checked.
3	API flow, program, two request inputs, responses and execution evidence		Code, actual request/response outputs and analysis to be checked.
4	Secure-access flow, four security considerations, threat and mitigation analysis		Authentication, authorization and secure-communication evidence to be checked.
5	Shared workspace, permission settings, two collaborative activities and contributions		Verify shared editing, user roles, permissions and contribution screenshots.
6	Integrated scenario, architecture, prototype, two user scenarios and results		Verify cloud access, collaboration workflow, tests and limitations.
7	Two initial proposals, comparison record, joint design, tests and cross-defence		Verify independent proposals, joint decisions, observed results and both students' understanding.

IMPLEMENTATION EVIDENCE CHAIN

Evidence item	Status	Observation
Cloud access methods		Browser, mobile and API access methods explained.
Cloud platforms		Two cloud platforms and their applications compared.
Cloud access demonstration		Actual screenshots need verification.
Web API implementation		Program and execution outputs need verification.
Cloud security		Authentication, authorization and security measures explained.
Cloud collaboration		Shared workspace and editing evidence need verification.
Mini-project		Design included; practical testing needs verification.
Day-4 design challenge		Individual proposals, joint design and validation need verification.
Student contributions		Contributions of both students need confirmation.
Screenshots		Original execution screenshots required.

SUBMISSION CHECKLIST (Not Separately Marked)

Requirement	Status
All 7 questions answered	☐ Yes ☐ Partly ☐ No
Cloud access methods explained	☐ Yes ☐ Partly ☐ No
Cloud architecture diagrams included	☐ Yes ☐ Partly ☐ No

Two cloud platforms compared	☐ Yes ☐ Partly ☐ No
Web API program and testing completed	☐ Yes ☐ Partly ☐ No
Cloud security explained	☐ Yes ☐ Partly ☐ No
Cloud collaboration demonstrated	☐ Yes ☐ Partly ☐ No
Mini-project completed and tested	☐ Yes ☐ Partly ☐ No
Day-4 design challenge completed	☐ Yes ☐ Partly ☐ No
Student A and B contributions recorded	☐ Yes ☐ Partly ☐ No
Screenshots and practical evidence attached	☐ Yes ☐ Partly ☐ No
Viva recommended	☐ Yes ☐ No

CLOUD COMPUTING AND BIG DATA ANALYTICS
ASSIGNMENT 3 – CO3

CO3: *Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications.*

Mode: Pair Work – 2 Students per Group
Duration: 1 Week **Total Marks:** 50

OBE-Based Evidence Principle

The assignment requires students to demonstrate measurable evidence of:
Understand → Design → Implement → Demonstrate → Analyse → Evaluate
Both students must understand the **complete assignment**. Students should not simply divide questions between them. Individual responsibilities are assigned within each question so that the contribution and attainment of CO3 can be independently verified.

Q1 – Accessing Cloud Services through Different Methods
6 Marks
Shared Group Work
Together, the pair must:
- Identify different methods of accessing cloud services.
- Study browser-based/cloud console access.
- Study access through mobile applications.
- Study access through APIs.
- Identify the role of Internet/network connectivity.
- Select one suitable cloud service for demonstration.

Student A – Access Method Design
Student A must individually:
- Identify and classify at least **three cloud access methods**.
- Prepare a simple architecture/flow diagram showing:

Client → Network → Cloud Service

- Explain how a user accesses the selected cloud service.
- Identify the hardware/software requirements.
- Explain the role of the client and network.
- Justify which access method is most suitable for the selected application.

Student B – Practical Evidence
Student B must individually:
- Demonstrate access to a cloud service using one selected method.
- Record the steps/screenshots as evidence.
- Test the access using different devices or access methods where possible.
- Record observations such as accessibility, response, and usability.
- Identify one limitation encountered during access.
- Explain how the limitation can affect cloud usage.

Cross-Verification
Student A must be able to explain Student B's practical demonstration.
Student B must be able to explain Student A's access architecture.
Evidence Expected: Architecture diagram + screenshots/observations + comparison.

Q2 – Cloud Platforms and Web Applications
7 Marks
Shared Group Work
Together:

- Select **two cloud platforms/services**.
- Identify the web applications available through them.
- Study how users interact with the applications.
- Identify the cloud resources/services supporting the application.
- Prepare a comparative observation.

Student A – Platform Analysis

Student A must individually:

- Identify the major components of the selected cloud platform.
- Explain how the platform supports web applications.
- Map the user request to the cloud service.
- Explain the role of the Internet, server/cloud infrastructure and application.
- Prepare a simple interaction diagram.

Student B – Application Investigation

Student B must individually:

- Select one real-world cloud-based web application.
- Demonstrate its major functions.
- Record the sequence of user actions.
- Identify where cloud access is involved.
- Collect screenshots/evidence of the demonstration.
- Analyse the benefits of using the cloud-based application.

Joint Evaluation

Together answer:

If the same application were installed and operated only on a local computer, what limitations might users experience compared with accessing it through the cloud?

Evidence Expected: Platform/application mapping + demonstration evidence + evaluation.

Q3 – Web APIs for Cloud Access

7 Marks

Shared Group Work

Together:

- Study the concept of a Web API.
- Identify how APIs enable application-to-cloud communication.
- Select a simple publicly accessible API or cloud API.
- Develop a small program/client to send a request.
- Observe and record the response.

Student A – API Design and Understanding

Student A must individually:

- Explain the purpose of the selected API.
- Identify the request and response components.
- Represent the API communication using a flow diagram.
- Explain the role of:
    - Client
    - API request
    - Cloud/server
    - API response
- Explain how APIs enable programmatic cloud access.

Student B – Implementation and Evidence

Student B must individually:

- Implement a simple API request using Python or another suitable language.
- Execute the program.
- Record input/request and output/response.
- Test at least **two different requests or inputs**.
- Capture execution evidence.
- Analyse the observed results.

Joint Analysis

Together determine:
Why is API-based access useful when a large number of users/applications need to communicate with a cloud service automatically?
Evidence Expected: Program + execution screenshots + request/response evidence + analysis.

Q4 – Cloud Security and Network Access
7 Marks
Shared Group Work
Together:
- Identify security issues involved in accessing cloud services.
- Study authentication and authorization.
- Study secure network communication.
- Identify possible threats during cloud access.
- Prepare a secure cloud-access scenario.

Student B – Security Design
Student B must individually:
- Identify at least **four security considerations** for cloud access.
- Explain authentication and authorization.
- Explain the importance of secure communication.
- Design a secure access flow:

User → Authentication → Network → Cloud Service → Authorization → Resource

- Identify possible points of attack.
- Suggest suitable security measures.

Student A – Security Investigation
Student A must individually:
- Investigate a practical cloud-access security scenario.
- Demonstrate or document a suitable authentication mechanism where feasible.
- Compare secure and insecure access situations conceptually.
- Record observations.
- Explain how poor security practices can affect cloud users and data.

Joint Evaluation
Together answer:
If cloud access is available from anywhere through the Internet, what additional security measures are necessary compared with a local application?
Evidence Expected: Secure-access architecture + threat identification + mitigation analysis.

Q5 – Collaboration Using Cloud-Based Services
6 Marks
Shared Group Work
Together:
- Select a cloud collaboration platform/service.
- Create a shared document/folder/project space.
- Configure appropriate sharing permissions.
- Perform collaborative editing.
- Record the collaboration process.

Student A – Collaboration Design
Student A must individually:
- Identify the collaboration requirements.
- Design a suitable cloud collaboration workflow.
- Explain:
    - Sharing
    - Concurrent access
    - Version control

    - Permission management
- Identify the roles of different users.
- Justify the selected sharing permissions.

Student B – Collaborative Implementation

Student B must individually:

- Participate in the shared cloud workspace.
- Perform at least two collaborative activities.
- Record evidence of contribution.
- Demonstrate sharing/permission settings.
- Observe how changes made by users are reflected.
- Identify one collaboration issue and suggest a solution.

Cross-Understanding

Student B must explain:

How does cloud collaboration reduce the need to exchange multiple copies of the same file?

Student A must explain:

What problems can occur if incorrect sharing permissions are given?

Evidence Expected: Shared workspace + contribution evidence + permission screenshots + observations.

Q6 – Integrated Cloud Access and Collaboration Mini Investigation
7 Marks

Students must design a small real-world scenario that requires **both cloud access and cloud collaboration**.

Suggested Scenarios

Choose **one**:

1. Cloud-based Student Project Management
2. Online Student Attendance and Faculty Collaboration
3. Cloud-Based Event Management
4. Cloud-Based Assignment Submission and Evaluation
5. Cloud-Based Hospital Appointment Coordination
6. Cloud-Based Disaster Information Sharing
7. Cloud-Based College Placement Coordination

Student A – System Design

Student A must individually:

- Define the problem.
- Identify users.
- Identify cloud access methods.
- Identify required cloud services.
- Design the system architecture.
- Explain how users access the system.
- Explain how users collaborate.

Student B – Evidence and Evaluation

Student B must individually:

- Develop a small prototype/mock-up/workflow where feasible.
- Demonstrate at least one cloud access mechanism.
- Demonstrate one collaboration mechanism.
- Record screenshots/evidence.
- Test the proposed solution using at least two user scenarios.
- Analyse the results.
- Identify limitations and possible improvements.

Joint Evaluation

Together answer:

How does combining cloud access and cloud collaboration improve the efficiency of the proposed system?

Q7 – DAY-4 DESIGN CHALLENGE

Design a Cloud-Based Collaborative Solution
10 Marks

This question must be solved **collaboratively**.

Neither student should independently prepare the complete solution and combine it later.

Stage 1 – Independent Design Proposal
Before discussing with each other:
Student A independently proposes:

- Problem definition
- Target users
- Cloud access method
- Required cloud services
- Collaboration mechanism
- Basic architecture

Student B independently proposes:

- Problem definition
- Target users
- Cloud access method
- Required cloud services
- Collaboration mechanism
- Basic architecture

Both initial proposals must be retained as evidence.

Stage 2 – Compare the Two Designs

Together discuss:

- Where the designs agree
- Where they differ
- Strengths and weaknesses
- Security concerns
- Accessibility
- Collaboration requirements
- Missing components
- Which ideas should be retained

Prepare a **Design Comparison Record**.

Stage 3 – Joint Final Design
Develop the final cloud-based solution.
The final design must identify:

- Problem
- Users
- Cloud access method
- Network requirements
- Cloud service/platform
- Web application/API requirement
- Collaboration mechanism

- Authentication/security
- Data/resource sharing
- Complete system workflow

Prepare an architecture such as:

Users → Client → Internet/Network → Cloud Platform → Application/API → Shared Resources

---

Stage 4 – Validation

Student A – Technical Validation

Student A must:
- Explain the selected cloud access method.
- Justify the platform/service selection.
- Explain the system architecture.
- Explain how API/web access is used, if applicable.
- Analyse the suitability of the design.

Student B – Experimental Validation

Student B must:
- Design test cases.
- Demonstrate the solution/workflow.
- Test different users/access conditions.
- Record observations.
- Present screenshots or other evidence.
- Compare expected and observed behaviour.

Stage 5 – Cross-Defence

Student A must explain:
What experimental evidence supports the group's cloud design?

Student B must explain:
Why was the selected cloud access and collaboration approach chosen?

Stage 6 – Joint Evaluation

Together determine:
- Benefits of the proposed cloud solution
- Access-related limitations
- Collaboration benefits
- Security risks
- Scalability considerations
- Possible improvements
- Situations where the proposed approach may not be appropriate

Final Question

What characteristics should a system designer consider before selecting cloud-based access and collaboration for a real-world application?

CONTRIBUTION RECORD

Each pair must include the following table at the end of the submission.

Question	Student A – Primary Contribution	Student B – Primary Contribution	Joint Work
Q1			
Q2			
Q3			
Q4			
Q5			
Q6			
Q7			

The contribution record must reflect the **actual work completed**.
Primary contribution does not mean exclusive responsibility. Both students are accountable for the complete assignment.

INDIVIDUAL VERIFICATION – OBE EVIDENCE
Faculty may select any question or component for individual verification.
Each student must be able to:

- Explain the cloud access method.
- Explain the role of client, network and cloud service.
- Explain web application/API access.
- Explain security considerations.
- Demonstrate cloud collaboration.
- Interpret experimental evidence.
- Justify design decisions.
- Respond to a modified requirement or scenario.

A student should not answer:
“My partner did this section.”
Understanding of the complete submitted work is mandatory.

RECOMMENDED FACULTY VERIFICATION MODEL – VIVA
For each student, conduct a brief **4–5 minute verification**.
Question 1 – Primary Responsibility
Ask one technical question from the component for which the student had primary responsibility.
Question 2 – Partner's Responsibility
Ask one question from a component primarily completed by the partner.
Question 3 – Transfer Question
Modify a condition and ask the student to reason about the consequence.
Examples:
“If Internet connectivity becomes unreliable, how would it affect cloud access?”

“If a shared cloud document is given editing permission to everyone, what problems could occur?”
“If the API becomes unavailable, how would your application behave?”
“If a user is authenticated but does not have permission to access a resource, what should happen?”
This provides **individual evidence of CO3 attainment** without requiring two completely separate assignments.

OBE Evidence Mapping

CO3 Component	**Assignment Evidence**
Build methods to access cloud	Q1, Q2, Q3
Access through platforms/web applications	Q2
Programmatic/API-based access	Q3
Secure cloud access	Q4
Collaboration within cloud	Q5
Apply cloud access + collaboration	Q6
Design and evaluate a cloud solution	Q7
Individual attainment	Viva + contribution record
Practical evidence	Screenshots, programs, observations, test results
Higher-order learning	Design, analysis, evaluation and transfer questions

Introduction

Cloud computing enables users to access applications, storage and computing resources through the Internet. Instead of depending only on software installed on one computer, users can access cloud-hosted services through browsers, mobile applications and web APIs. Cloud collaboration allows multiple users to work on shared documents, tasks and information with controlled permissions.

This assignment studies different cloud access methods, compares cloud platforms and web applications, demonstrates API-based access, analyses cloud security, and designs a collaborative student project-management solution. The same concepts are combined in the Day-4 design challenge.

Q1 – Accessing Cloud Services through Different Methods (6 Marks)

1.1 Cloud access methods

Access method	How it works	Example / suitable use	Advantages	Limitations
Browser-based access	A browser connects to a cloud-hosted application through HTTPS.	Google Drive or Microsoft 365 web app; document editing.	No local installation; works across devices.	Requires a compatible browser and reliable Internet.
Mobile application access	A mobile app communicates with cloud services using secure network requests.	Google Drive mobile app; upload and view files.	Convenient notifications, camera/file integration and mobility.	Device storage, battery, app permissions and network limitations.
API-based	A program sends	Python client	Automation,	Requires

access	structured HTTP requests to a cloud/API endpoint and receives responses.	calling a REST API.	system integration and bulk operations.	programming, authentication and API-rate-limit handling.

1.2 Access architecture

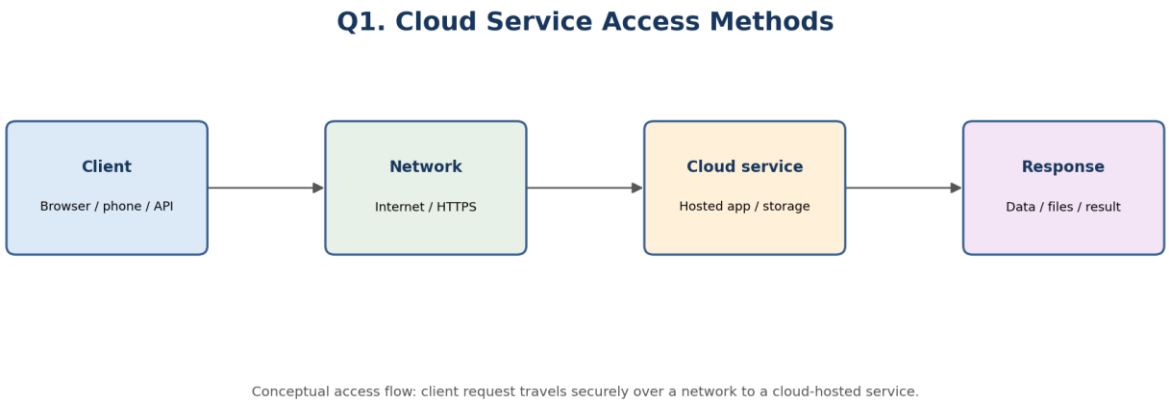


Figure 1. Client–network–cloud access architecture.

The client is the device or software used to initiate a request. The network carries the request to the cloud service. The cloud service processes the request and returns the result. HTTPS protects data while it travels between the client and service.

1.3 Demonstration plan and observations

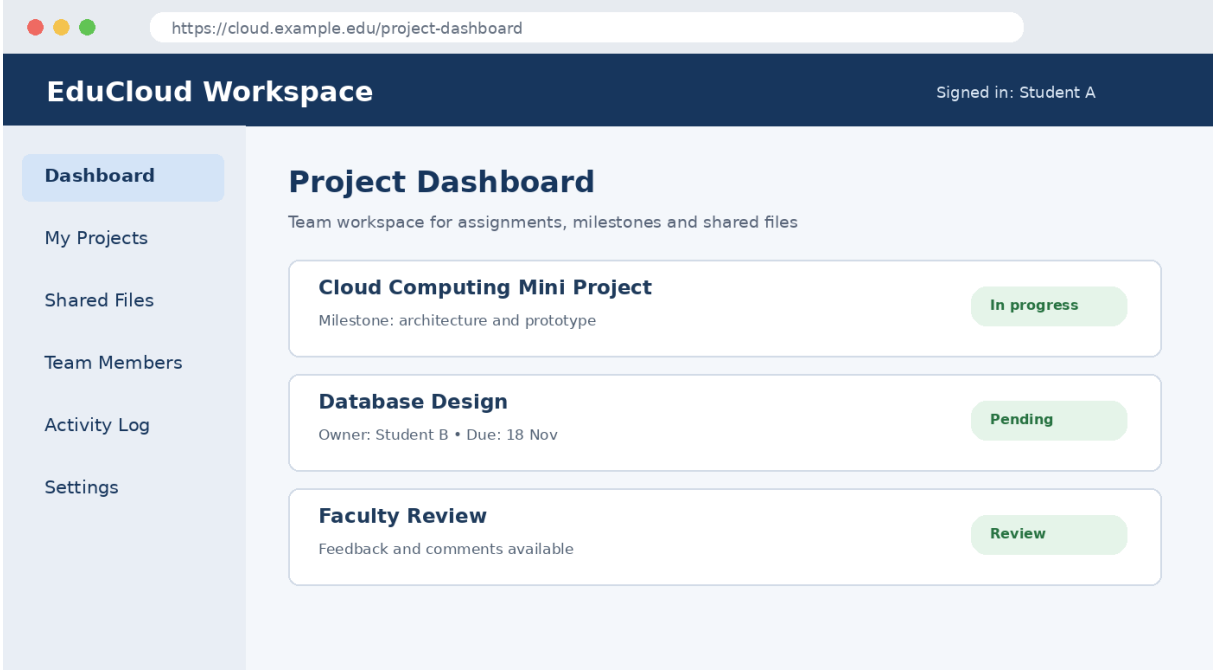


Figure 8. cloud project dashboard mockup

1. Open a test document or upload a non-confidential sample file.
2. Repeat the access using the official mobile application, if available.
3. Record the time taken to open the file, ease of navigation and any access limitation.
4. For API access, use a public test endpoint and a small Python client as demonstrated in Q3.

Observation criterion	**Browser**	**Mobile app**	**API client**
User interaction	Visual interface and mouse/keyboard	Touch interface	Programmatic request
Installation	Browser required	App installation required	Python/runtime and libraries
Best suited for	Manual access and editing	On-the-go access	Automation and integration
Evidence to capture	Signed-in service and sample file view	App view and same sample file	Code, request and returned response

Most suitable method: browser access is suitable for collaborative document editing because it offers a full interface without requiring local installation. API access is more suitable for automatic integration between applications.

Q2 – Cloud Platforms and Web Applications (7 Marks)

2.1 Selected platforms

Platform	Cloud services / components	Example web application	How it supports the application
Google Cloud Platform (GCP)	Compute Engine, App Engine/Cloud Run, Cloud Storage, Cloud SQL, IAM and networking.	A web-based student project tracker hosted on Cloud Run with Cloud SQL storage.	Provides application hosting, database, object storage, identity and network services.
Microsoft Azure	Azure App Service, Azure Virtual Machines, Azure Blob Storage, Azure SQL Database, Microsoft Entra ID and networking.	A web-based task and document collaboration application hosted on App Service.	Provides managed web hosting, data storage, authentication and monitoring.

2.2 User interaction flow

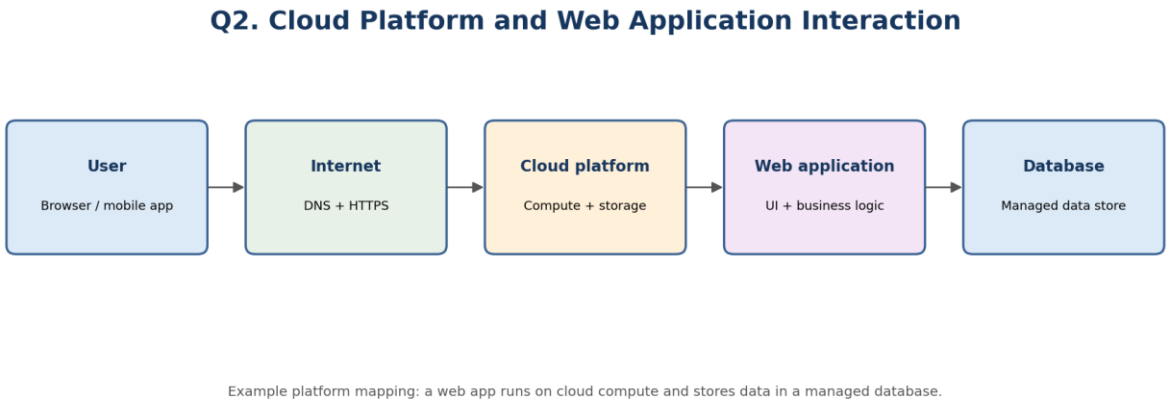


Figure 2. User-to-application-to-database interaction on a cloud platform.

When a user enters a web address, the browser resolves the service address and establishes an HTTPS connection. The cloud-hosted application receives the request, applies its business logic and accesses the database or storage service if required. The result is returned to the user's browser.

2.3 Comparison

Factor	GCP	Microsoft Azure
Application hosting	Cloud Run, App Engine and Compute Engine.	Azure App Service and Virtual Machines.
Storage	Cloud Storage and managed databases.	Blob Storage and Azure SQL.
Identity	Cloud IAM and identity integrations.	Microsoft Entra ID and Azure RBAC.
Typical strength	Integration with Google cloud-native data and application services.	Integration with Microsoft enterprise tools and identity services.

2.4 Cloud versus local installation

If the same application is installed and operated only on a local computer, users may face limited access from other locations, difficulty sharing current versions, manual backups and dependence on that computer's availability. A cloud-hosted version supports access from multiple devices, centralised updates, shared data and easier scaling. However, it depends on Internet connectivity and requires careful access control and ongoing cloud-service cost management.

Q3 – Web APIs for Cloud Access (7 Marks)

3.1 Selected API

A Web API allows one software application to communicate with another using standard web protocols. This example uses the public JSONPlaceholder demonstration API, which provides sample post records. Endpoint: https://jsonplaceholder.typicode.com/posts. It is intended for testing and learning, not for storing real student information.

3.2 API request–response flow

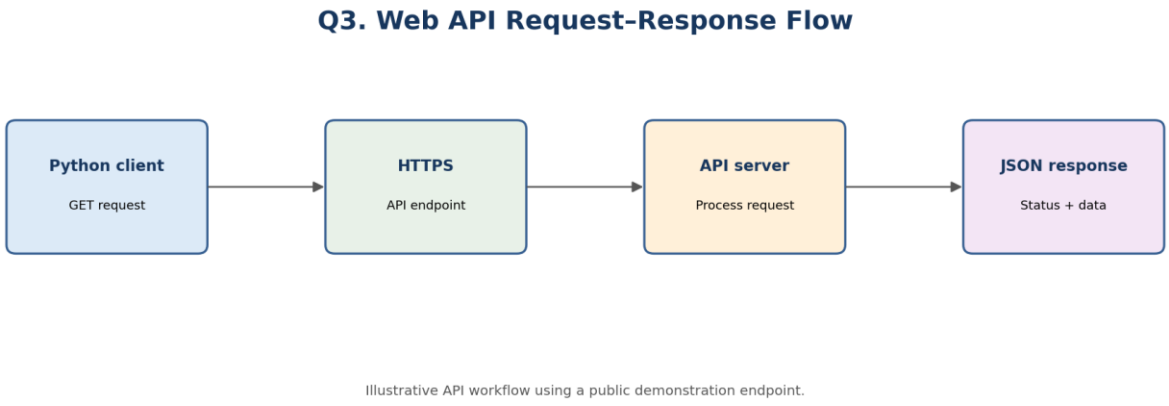


Figure 3. API client sends an HTTPS request and receives a JSON response.

3.3 Python API client

```
import requests
```

```
url = "https://jsonplaceholder.typicode.com/posts"

# Request 1: retrieve the first sample post
response1 = requests.get(url + "/1", timeout=10)
print("Request 1 status:", response1.status_code)
print("Request 1 response:", response1.json())

# Request 2: retrieve the second sample post
response2 = requests.get(url + "/2", timeout=10)
print("Request 2 status:", response2.status_code)
print("Request 2 response:", response2.json())
```

Installation and execution: install Python, open a terminal and run `pip install requests`. Save the program as `cloud_api_demo.py` and execute `python cloud_api_demo.py`. Internet access is required.

3.4 Illustrative response format



Figure 9. terminal-output mockup showing the expected API response format; verify by running the Python program.

The following is an example of the response structure expected from the public test API. It is illustrative and should be checked by running the code.

```
Request 1:
HTTP status: 200
{
  "userId": 1,
  "id": 1,
  "title": "sunt aut facere repellat provident",
  "body": "sample post body ..."
}

Request 2:
HTTP status: 200
{
  "userId": 1,
  "id": 2,
  "title": "qui est esse",
  "body": "sample post body ..."
}
```

3.5 Analysis

The client identifies the API endpoint and sends an HTTP GET request. The server processes the request and returns a status code and JSON data. A status code of 200 indicates that the request was successful. APIs are useful when many users or applications need automatic communication because software can exchange structured data without manual copying. APIs also support integration, automation and reusable services. Production APIs should additionally use suitable authentication, authorisation, validation, rate limiting and monitoring.

Q4 – Cloud Security and Network Access (7 Marks)

4.1 Secure access architecture

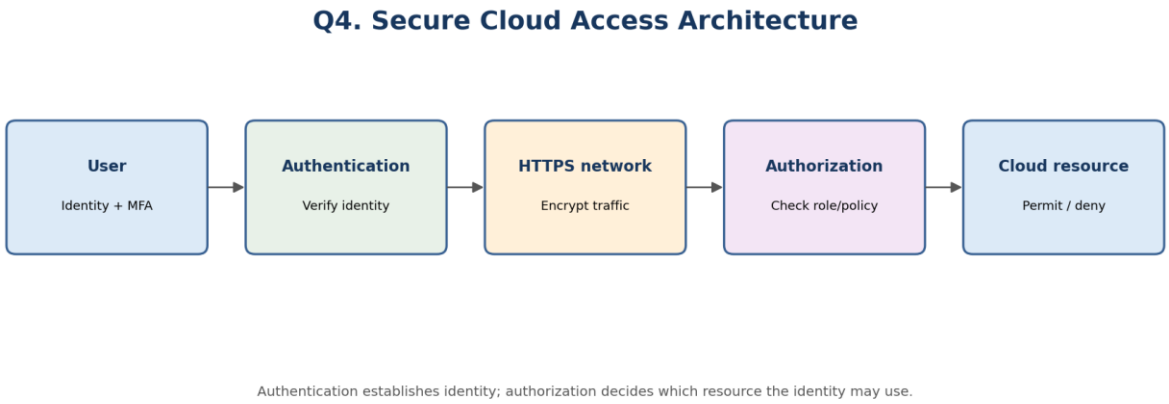


Figure 4. Secure cloud-access flow with authentication and authorisation.

4.2 Security considerations and threats

Security consideration / threat	Explanation	Mitigation
Authentication	Verifies the identity of a user before access.	Strong passwords, multi-factor authentication (MFA), single sign-on and secure account recovery.
Authorisation	Determines which resources an authenticated user can access.	Role-based access control (RBAC), least privilege and periodic permission reviews.
Data interception	Unencrypted traffic may be read or modified in transit.	HTTPS/TLS, valid certificates and secure network configuration.
Account compromise /	Attackers may steal credentials and access cloud	MFA, phishing awareness, login alerts and conditional access.

phishing	resources.	
Data leakage / incorrect sharing	Public links or excessive permissions may expose confidential files.	Private-by-default sharing, expiry dates, restricted domains and audit logs.
Availability attacks	Traffic floods or service failures may make applications unavailable.	Backups, monitoring, rate limiting, recovery plans and provider availability controls.

4.3 Authentication and authorisation

Authentication answers: 'Who are you?' For example, a user signs in with an account and a second verification factor. Authorisation answers: 'What are you allowed to do?' For example, a student may view and edit their own project tasks, while a faculty member may review all project submissions.

4.4 Secure and insecure access

Insecure situation	**Secure alternative**
Using an open public link for confidential files.	Share with named accounts and restrict access to required users.
Sending credentials over an unencrypted connection.	Use HTTPS/TLS and never share passwords.
Giving every user administrator permissions.	Apply least privilege and role-based permissions.
Using one shared account for a group.	Give each member an individual account and maintain activity logs.

Poor security practices can expose personal information, allow unauthorised modification, cause loss of trust and interrupt access to cloud services. Internet-based access requires

stronger identity verification, encryption, monitoring and permission management than an isolated local application.

Q5 – Collaboration Using Cloud-Based Services (6 Marks)

Selected service: Google Drive with Google Docs. It provides a shared cloud workspace where two students can create, edit, comment on and review the same document. Microsoft OneDrive with Word Online is an equivalent alternative.

5.1 Collaboration workflow

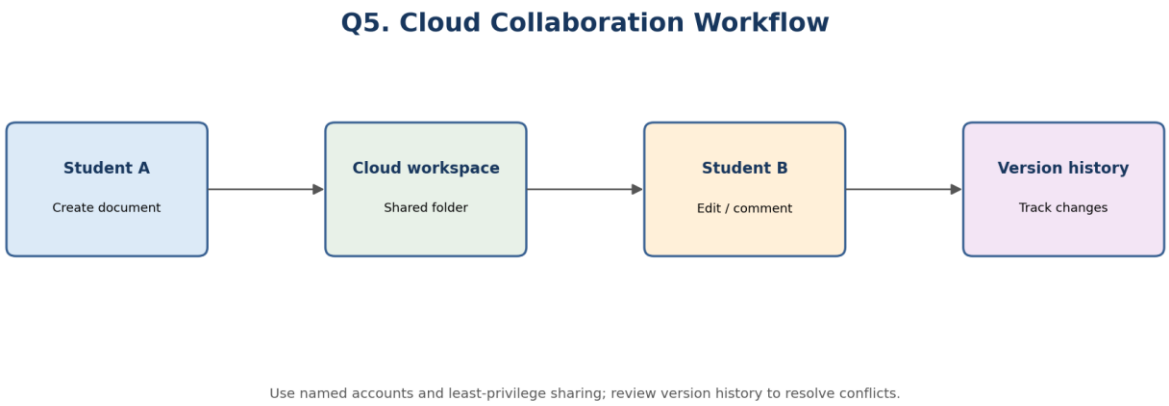


Figure 5. Shared document workflow with version history.

5.2 Roles and permissions

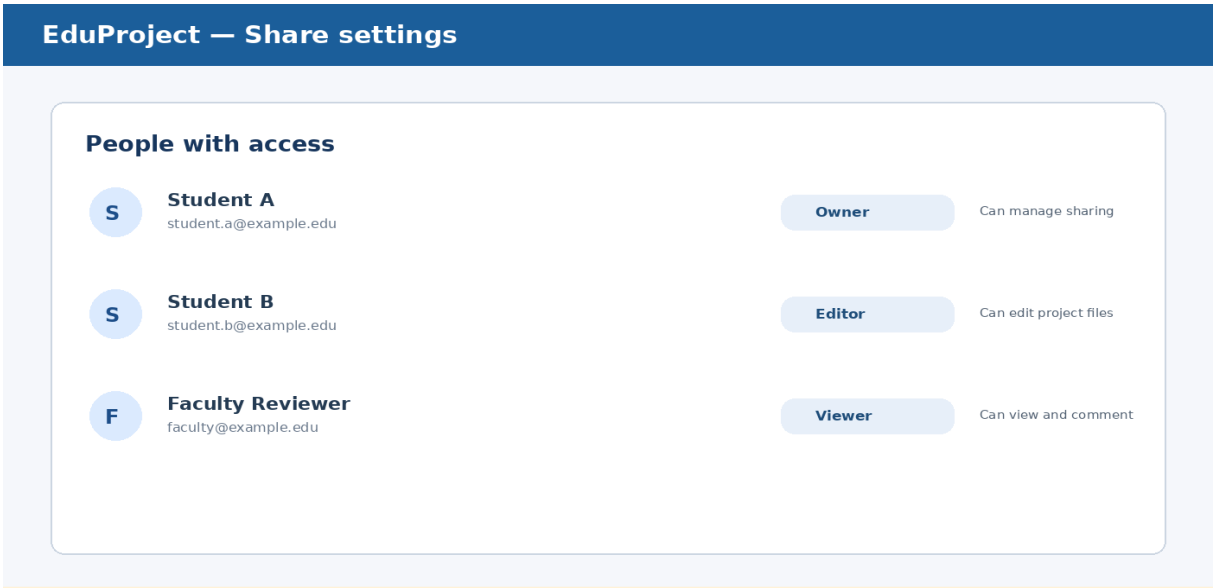


Figure 10 sharing-permissions interface; configure actual roles in the selected cloud service.

Role	Permission	Reason
Student A – owner	Owner / manage sharing	Creates the shared folder and controls access.
Student B – collaborator	Editor	Can add content, make corrections and comment.
Faculty – reviewer	Viewer or commenter	Can review work without unrestricted editing.

5.3 Two collaborative activities

5. Activity 1 – Concurrent editing: Student A creates a document and shares it with Student B as an editor. Both students add separate sections and verify that changes appear in the shared document.
6. Activity 2 – Comments and version history: Student B adds a comment or suggestion. Student A responds, resolves the comment and checks version history to identify earlier edits.

5.4 Benefits and possible issue

Cloud collaboration reduces multiple file copies because everyone works on one shared version. It supports real-time editing, comments, version history and access from different devices. A possible issue is accidental overwriting or unauthorised editing. This can be reduced through named accounts, suitable roles, comments, version history and regular review of sharing permissions.

Evidence to capture: the shared folder/document, sharing settings showing Student A and Student B roles, an edit made by each student, a comment, and version history. These must be captured from the actual collaboration session.

Q6 – Integrated Cloud Access and Collaboration Mini Investigation (7 Marks)

6.1 Scenario and problem definition

Selected scenario: Cloud-Based Student Project Management. Student teams often manage project topics, tasks, deadlines, files and faculty feedback using separate messages and document copies. This can cause missed deadlines, duplicated work and difficulty tracking progress. The proposed system provides one shared project workspace.

6.2 Users and requirements

User	Main activities
Student	Sign in, view assigned tasks, update status, upload project files and comment.
Team leader	Create tasks, assign members, set deadlines and monitor progress.
Faculty	Review project status, provide feedback and view submitted files.
Administrator	Manage accounts, roles, access policies and system availability.

6.3 Architecture

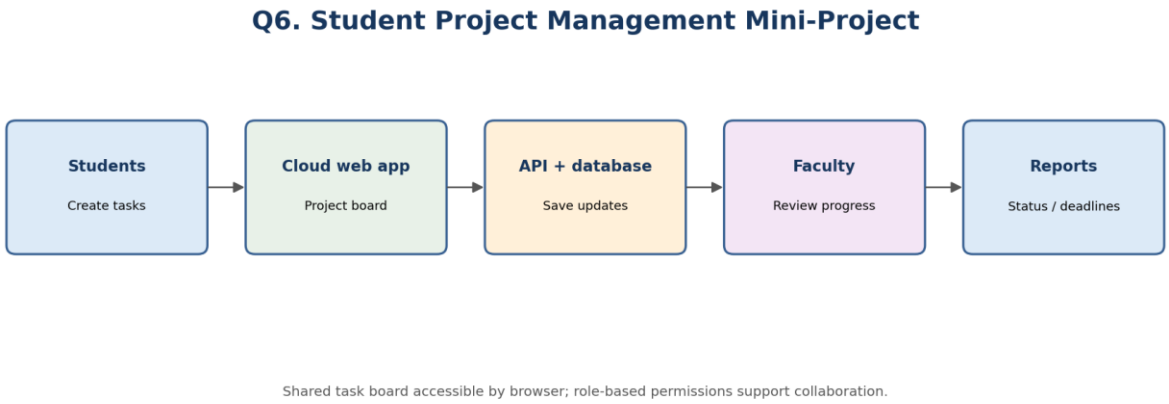


Figure 6. Integrated student project-management cloud architecture.

6.4 Required cloud services

- Cloud application hosting for the web interface and business logic.
- Managed database to store users, projects, tasks, deadlines and status.
- Cloud object storage for project documents and supporting files.
- Web APIs for task creation, updates, file metadata and status retrieval.
- Identity and access management for sign-in and role-based permissions.
- Monitoring, backup and audit logs for reliability and accountability.

6.5 Prototype workflow

7. A student signs in through a browser using an individual account.
8. The team leader creates a project and adds tasks with owners and due dates.
9. Team members update task status and upload files to the shared workspace.
10. Faculty accesses the project dashboard, reviews progress and adds feedback.
11. The application stores changes in the cloud database and records relevant activity.

6.6 Two user test scenarios and evaluation



Figure 11. validation dashboard; test statuses are planned and require actual execution.

Test scenario	Expected behaviour	Evaluation criterion
Scenario 1: Student updates a task from a browser.	Task status is saved and visible to authorised team members.	Verify update, refresh the page and confirm the saved status.
Scenario 2: Faculty opens the project dashboard.	Faculty can view project progress and provide feedback; cannot change administrator settings.	Verify role permissions and confirm restricted actions are denied.

Expected benefit: Combining cloud access and collaboration makes project information available to authorised users from different locations and keeps tasks, files and feedback in one shared workspace. Limitations include Internet dependency, cloud cost, account security and the need for user training. Improvements include mobile-friendly design, notifications, offline draft support and analytics.

Q7 – Day-4 Design Challenge: Design a Cloud-Based Collaborative Solution (10 Marks)

Stage 1 – Independent design proposals

Student A proposal – Centralised project workspace: Student A proposes a browser-based project portal hosted on a cloud platform. Students create tasks, assign owners and upload files. Faculty reviews the shared project dashboard. The proposal prioritises a simple web interface, managed database and role-based access.

Student B proposal – Collaboration-first task and document hub: Student B proposes a shared workspace built around cloud documents, task boards and file storage. Students collaborate through comments and status updates, while faculty receives read/review access. The proposal prioritises real-time editing, version history and notifications.

These are example proposal structures. The pair should retain their actual independent proposals and adjust the wording to reflect what each student independently suggested.

Stage 2 – Design comparison record

Comparison area	Student A proposal	Student B proposal	Joint decision
Main focus	Central project portal	Shared workspace and task collaboration	Combine portal with shared collaboration features.
Access method	Browser-based access	Browser and cloud collaboration app	Browser first; mobile access as an extension.
Cloud services	Application hosting + database	Cloud storage + document/task services	Use hosting, database, storage and APIs.
Collaboration	Task assignment and progress	Concurrent editing, comments and	Include tasks, comments, shared

	dashboard	version history	files and history.
Security	Role-based access	Named users and restricted sharing	Apply MFA, RBAC, HTTPS and audit logs.

Stage 3 – Joint final design

Final solution: EduProject Cloud – a cloud-based student project coordination and collaboration portal. It allows students and faculty to access project information securely through a browser and collaborate using shared tasks, documents, comments and progress reports.

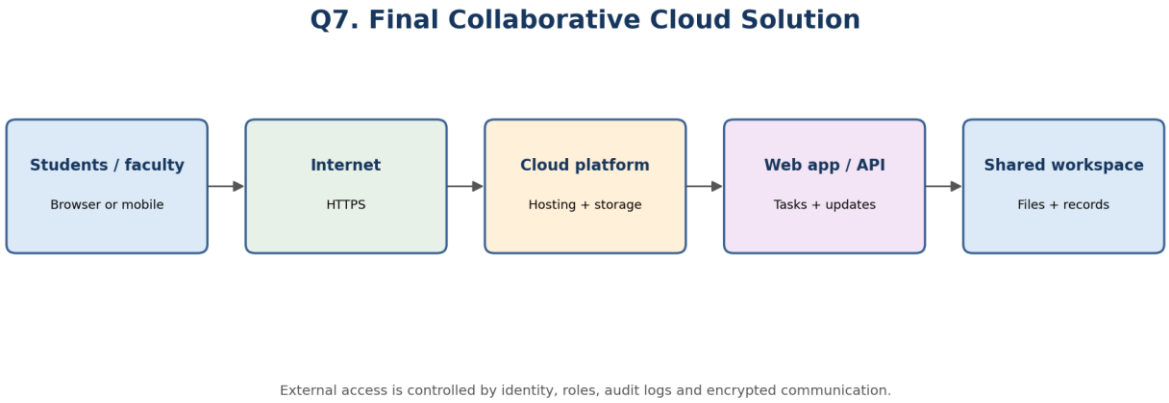


Figure 7. Joint final cloud-based collaborative solution.

3.1 Complete workflow

12. The user opens the portal using a browser or supported mobile client.
13. The system authenticates the user and checks their assigned role.
14. The web application sends requests to its backend APIs over HTTPS.
15. The API validates the request and reads or updates the cloud database.
16. Files are stored in cloud object storage with access permissions.
17. The updated task, comment or file information is displayed to authorised collaborators.
18. Audit logs and version history help trace changes.

3.2 Security and resource-sharing design

- Use HTTPS/TLS for all client-to-cloud communication.
- Enable MFA for faculty and administrators; use individual student accounts.
- Apply role-based access control and least-privilege permissions.
- Keep project folders private by default and share only with named users.
- Validate API inputs, restrict request rates and protect credentials/secrets.
- Maintain backups, audit logs and version history.

Stage 4 – Validation plan

Test ID	**Student A – technical validation**	**Student B – experimental validation**	**Expected result**
T1 – Sign-in	Explain browser-to-cloud HTTPS and identity flow.	Test sign-in using an authorised test account.	Valid user reaches the dashboard.
T2 – Task update	Explain API request and database update.	Create a task and change its status.	Updated status persists and is visible to authorised members.
T3 – Permission check	Explain RBAC policy for student and faculty.	Attempt a restricted action with a student account.	Unauthorised action is denied.
T4 – Collaboration	Explain shared workspace and version history.	Two users edit/comment on a shared test document.	Changes and comments are visible in the shared workspace.

Record actual observations for each test, including date, account role, action, expected result, observed result and screenshot reference. Do not mark a test as passed until it has been performed.

Stage 5 – Cross-defence

Student A should explain the evidence that supports the design: successful authorised access, API responses, saved task updates, role restrictions and shared document changes. Student B should explain why browser access, cloud hosting, APIs and shared workspace services were selected: they support multi-device access, structured communication, central storage and collaboration.

Stage 6 – Joint evaluation

Evaluation factor	Benefits	Limitations / improvements
Access	Browser access from different locations and devices.	Internet dependency; improve with responsive design and carefully controlled offline drafts.
Collaboration	Shared tasks, comments, files and version history.	Concurrent edits may conflict; use version history and clear task ownership.
Security	MFA, HTTPS, RBAC and audit logs reduce unauthorised access.	Misconfigured sharing remains a risk; review permissions regularly.
Scalability	Cloud hosting and managed services can scale with user demand.	Higher usage may increase cost; use monitoring and budgets.
Reliability	Managed backups and monitoring support recovery.	Provider or network outage may interrupt service; define

		recovery procedures.

Final answer

Before selecting cloud-based access and collaboration, a system designer should consider user requirements, accessibility, network reliability, application compatibility, data sensitivity, authentication, authorisation, encryption, collaboration needs, storage capacity, scalability, service availability, backup and recovery, operating cost and vendor dependence. The chosen design should provide the required functionality while protecting data and remaining manageable as the number of users grows.

Contribution Record

The assignment brief requires the contribution record to reflect the work actually completed by both students. Complete this table together; do not claim work that was not performed.

Question	**Student A – primary contribution**	**Student B – primary contribution**	**Joint work**
Q1	Access-method classification and architecture	Access demonstration and observations	Compare access methods
Q2	Platform analysis and interaction mapping	Web application investigation and screenshots	Compare cloud and local use
Q3	API purpose, flow and request/response explanation	Python client execution and output capture	Analyse API usefulness
Q4	Security	Security design,	Evaluate secure

	investigation and observations	threats and mitigations	access
Q5	Collaboration workflow and permissions design	Shared workspace edits and contribution evidence	Review collaboration issues
Q6	Scenario and system architecture	Prototype/workflow tests and evaluation	Assess integrated solution
Q7	Technical validation and design rationale	Experimental validation and observations	Compare proposals and defend final design

Conclusion

The assignment demonstrates how cloud services can be accessed through browsers, mobile applications and APIs. Cloud platforms provide hosting, storage, databases and identity services for web applications. Secure access requires authentication, authorisation, encryption and permission management. Cloud collaboration supports shared editing, comments and version history. The proposed EduProject Cloud solution combines these capabilities to improve student project coordination and faculty review. Its success should be validated through actual access, API, permission and collaboration tests.

References

- Google Cloud documentation: https://cloud.google.com/docs
- Microsoft Azure documentation: https://learn.microsoft.com/azure/
- Google Drive Help: https://support.google.com/drive/
- JSONPlaceholder public demonstration API: https://jsonplaceholder.typicode.com/

- Python Requests documentation: https://requests.readthedocs.io/

Appendix A – Detailed Requirements and User Stories

A clear requirements baseline helps ensure that the proposed EduProject Cloud system solves the correct problem. Functional requirements describe what the system must do. Non-functional requirements describe the quality levels and constraints that must be maintained while it performs those functions.

Requirement ID	Type	Requirement statement	Acceptance evidence
FR-01	Functional	A student can sign in using an individual account.	A valid test account reaches the student dashboard.
FR-02	Functional	A team leader can create a project and assign tasks to members.	A new task is saved with an owner and due date.
FR-03	Functional	A team member can update the status of an assigned task.	The new status remains visible after refresh.
FR-04	Functional	A user can upload and access authorised project files.	An uploaded test file is available to permitted collaborators.
FR-05	Functional	A faculty reviewer can view progress and provide feedback.	Reviewer can view the project and submit a comment.

NFR-01	Security	Users must access only information allowed by their role.	A restricted action is denied for an unauthorised role.
NFR-02	Availability	The application should have a documented backup and recovery process.	A backup and restoration procedure is reviewed.
NFR-03	Usability	The interface should be understandable to students and faculty.	Representative users complete basic tasks.
NFR-04	Performance	Common dashboard actions should return within an agreed response-time target.	Response times are measured during user testing.

Appendix B – Cloud Deployment and Data Design

The proposed system uses a three-layer design. The presentation layer provides the browser interface. The application layer validates requests and applies project rules. The data layer stores structured project information and uploaded files. Separating these responsibilities makes the system easier to maintain and allows individual components to scale independently.

Layer	**Main components**	**Responsibility**	**Design consideration**
Presentation	Responsive web	Displays projects,	Use accessible

	pages and dashboard	tasks, comments and reports.	forms and clear status indicators.
Application	Web service and REST API	Authenticates requests, validates input and applies business rules.	Keep secrets outside source code; validate all inputs.
Data	Managed relational database	Stores users, teams, projects, task status and comments.	Use backups, access controls and database constraints.
Object storage	Cloud file bucket/container	Stores documents and project attachments.	Use private access and short-lived authorised links.
Operations	Monitoring, logs and backup services	Tracks errors, availability and recovery readiness.	Alert on unusual activity and backup failures.

A simple relational data model can contain the following entities:

Entity	**Example attributes**	**Relationship**
User	user_id, name, email, role	A user can belong to one or more teams.
Team	team_id, team_name	A team can own several projects.
Project	project_id, title, team_id, faculty_id	A project belongs to a team and has a reviewer.
Task	task_id, project_id, assignee_id, due_date, status	A project contains multiple tasks.

Comment	comment_id, task_id, author_id, message, created_at	A task can have multiple comments.
FileRecord	file_id, project_id, uploader_id, storage_key	A project can reference multiple stored files.

Structured information such as task status belongs in the database, while the actual uploaded file content belongs in object storage. The database stores a storage key and metadata rather than placing large files directly into a task record. This separation improves retrieval and backup management. Database constraints should prevent invalid references, such as assigning a task to a user who is not part of the project team.

Appendix C – API Design, Validation and Error Handling

A REST-style API exposes application functions through resource URLs and HTTP methods. GET retrieves information, POST creates a resource, PUT or PATCH updates a resource, and DELETE removes a resource where the user's role permits it. The API should return meaningful status codes and a consistent JSON structure.

Method	Example endpoint	Purpose	Typical response
GET	/api/projects	List projects visible to the user.	200 OK with JSON list.
GET	/api/projects/15/tasks	Retrieve tasks for project 15.	200 OK with task records.
POST	/api/projects/15/tasks	Create a task in project 15.	201 Created with new task ID.
PATCH	/api/tasks/82	Update task 82 status.	200 OK with updated record.
DELETE	/api/tasks/82	Delete task if policy permits.	204 No Content or 403 Forbidden.

The Q3 demonstration uses a public sample API to teach request and response handling. In the real project, the API would use authenticated endpoints and must not expose private student records through a public demonstration service.

A robust client should handle network failures, timeouts, invalid JSON and unsuccessful HTTP status codes. The following example shows a safer request pattern for the demonstration API:

```
import requests

url = "https://jsonplaceholder.typicode.com/posts/1"
```

```
try:
    response = requests.get(url, timeout=10)
    response.raise_for_status()
    post = response.json()
    print("Request successful")
    print("Post ID:", post.get("id"))
    print("Title:", post.get("title"))
except requests.exceptions.Timeout:
    print("The request timed out")
except requests.exceptions.RequestException as error:
    print("API request failed:", error)
except ValueError:
    print("The server response was not valid JSON")
```

A timeout prevents a client from waiting indefinitely. raise_for_status() treats unsuccessful HTTP status codes as errors. The exception handlers provide a controlled response instead of allowing the program to stop unexpectedly. Production systems should also log diagnostic information without recording passwords, access tokens or confidential response data.

Appendix D – Security Policy and Data Protection Controls

EduProject Cloud may contain student names, email addresses, project documents, feedback and academic progress information. The system should therefore collect only the information required for project coordination and restrict access according to each user's responsibilities.

Control area	Policy / implementation	Reason
Identity	Unique account for every student, faculty member and administrator.	Provides accountability and avoids shared credentials.
MFA	Require an additional verification factor for privileged roles.	Reduces risk from stolen passwords.
RBAC	Student, team leader, faculty reviewer and administrator roles.	Limits actions to the user's responsibility.
Transport security	Use HTTPS/TLS for all application and API traffic.	Protects information in transit.
Storage security	Private storage by default; encryption at rest where supported.	Reduces unauthorised disclosure risk.
Audit logging	Record sign-ins, permission changes, file sharing and important updates.	Supports investigation and accountability.
Backup	Maintain scheduled backups and periodically test restoration.	Supports recovery from accidental deletion or service failure.
Retention	Define how long project	Avoids keeping personal

	files and logs are retained.	information longer than necessary.

A privacy-aware design should display a clear purpose for collecting each field. For example, a phone number should not be collected if email is sufficient for project communication. Access to files should be limited to named project members and reviewers. Public sharing links should be disabled by default or tightly controlled. When a student leaves a team, their access should be reviewed and removed where appropriate.

Security is a continuous process. The application should be updated regularly, dependencies should be checked for known vulnerabilities, and administrators should review unusual sign-in activity. A security incident procedure should identify who receives an alert, how access is restricted, how evidence is preserved and how affected users are informed under applicable institutional policy.

Appendix E – Collaboration Governance and Conflict Resolution

A shared cloud workspace is effective only when collaborators understand ownership, editing rights and communication practices. The team should agree on a simple governance model before working on the same files.

Situation	Recommended practice	Expected outcome
Creating a shared file	Owner creates the file in the project folder and shares it with named accounts.	A single authorised source of truth.
Concurrent editing	Use separate headings or assigned sections and communicate before major restructuring.	Fewer conflicting edits.
Review and feedback	Use comments or suggestions rather than silently replacing another person's work.	Clear review trail.
Accidental deletion	Check version history and restore the correct earlier version.	Reduced risk of permanent work loss.
Member leaves team	Owner or administrator reviews and removes access.	Access remains limited to current members.
Final submission	Agree a file naming convention and mark one version as final.	Avoids submitting an outdated copy.

Suggested naming convention: Course_Assignment_Team_Document_Version_Date. For example, CSA15_CO3_Team01_Report_v2_2026-11-05. Team members should avoid names such as final-final-new because they make version selection unclear.

A practical contribution record should state the work actually performed by each student. Student A may take responsibility for requirements, API explanation and technical design, while Student B may take responsibility for platform investigation, collaboration testing and evidence organisation. Both students should jointly review security, evaluation and final recommendations. The roles can be exchanged; the record must match the real division of work.

If a disagreement occurs, the team should first compare the requirement and evidence, then discuss alternatives and record the agreed decision. Version history provides a record of changes, but it does not replace communication. A short weekly review can confirm completed tasks, blockers, upcoming deadlines and changes to scope.

Appendix F – Implementation Roadmap, Risks and Resource Planning

Phase	Activities	Deliverable	Main risk / control
1. Requirements	Confirm user roles, project workflow, essential features and data fields.	Approved requirements list.	Unclear scope; obtain faculty and student agreement.
2. Design	Prepare architecture, database model, API routes and access policy.	Design specification and diagrams.	Over-complex design; prioritise essential features.
3. Prototype	Build sign-in, project list, task creation and status update.	Working minimum viable prototype.	Integration issues; test each component early.
4. Collaboration	Add file sharing, comments, reviewer access and version history.	Shared project workspace.	Accidental exposure; use private-by-default permissions.
5. Validation	Run functional, permission, API and usability tests.	Test record and issue list.	False confidence; record actual observed outcomes.
6. Deployment	Configure hosting, database, storage, domain and monitoring.	Deployed application.	Configuration errors; use a deployment checklist.
7. Operation	Monitor, patch,	Operational review	Service outage or

	back up and review user access.	record.	data loss; test recovery.

Resource planning should include application hosting, database capacity, file storage, outbound network usage, monitoring and backup. A small student prototype can begin with a free or low-cost development tier where available, but free-tier eligibility and limits depend on the provider and may change. Production deployment should use an approved institutional account and budget.

A risk register should assign each risk an owner, likelihood, impact and response. High-priority risks include accidental public file sharing, compromised accounts, loss of uploaded work, database unavailability and unexpected cloud charges. Controls should be reviewed during each project milestone rather than only at the end.

Appendix G – Expanded Validation Plan, Metrics and Glossary

Test category	Test method	Metric / evidence	Pass condition
Functional	Create project, assign task, update status and add comment.	Saved records and visible status after refresh.	All expected actions work for an authorised user.
Access control	Attempt student, faculty and administrator actions with each role.	Allowed/denied action log.	Only permitted actions succeed.
API	Send valid, invalid and unauthorised requests.	HTTP status, response structure and error handling.	Correct response and safe error handling.
Usability	Ask representative users to complete a task without assistance.	Completion rate and user feedback.	Users can complete core workflow clearly.
Performance	Measure dashboard and task update response time under agreed load.	Response time and error rate.	Results meet the response-time target agreed by the project.
Recovery	Restore a test backup in a controlled environment.	Restoration record and data integrity check.	Required records are restored and verified.

Useful evaluation metrics include task completion rate, API success rate, median response time, p95 response time, number of unauthorised actions correctly denied, backup restoration time and user satisfaction. A metric should be defined before testing so that the team does not choose a success criterion after seeing the result.

Glossary:

Term	**Meaning**
API	Application Programming Interface; a defined way for software systems to communicate.
Cloud hosting	Running an application on computing resources provided through a cloud service.
HTTPS	HTTP protected by TLS encryption for communication between client and server.
Authentication	The process of verifying a user's identity.
Authorisation	The process of deciding what an authenticated user is allowed to do.
RBAC	Role-Based Access Control; permissions are assigned through roles.
REST	An architectural style commonly used to design web APIs around resources and HTTP methods.
JSON	JavaScript Object Notation; a lightweight data format commonly used in API responses.
Version history	A record of earlier file versions that can help review or restore changes.

MFA	Multi-Factor Authentication; sign-in requires more than one verification factor.

Overall evaluation: The proposed solution is appropriate for a student project because it combines common cloud capabilities into a manageable workflow. Its effectiveness should be judged using the defined requirements and actual test evidence. Design diagrams and sample output explain the intended operation, while only completed tests can establish that the implementation behaves as expected.